

Nama: Mohammad Isa Widiyanto
NPM: 714240013
D4 Teknik Informatika 2A
Pengantar Strategi Algoritma

LAPORAN ANALISIS ALGORITMA SMARTBANK

Dokumen ini merangkum hasil uji coba empiris terhadap tiga algoritma utama yang diterapkan pada ekosistem **SmartBank** (KMP, Merge Sort, dan Greedy) dibandingkan dengan pendekatan konvensional (Brute Force, Bubble Sort, Iterasi Naif). Pengujian ini dilakukan dengan menyuntikkan **5.000 data transaksi acak (Dummy Data)** ke dalam basis data.

1. Algoritma Pencarian KMP vs Brute Force (Ledger Admin & Riwayat Nasabah)

Pengujian dilakukan dengan mencari sebuah kata kunci pendek (contoh: DUMMY) pada 5.000 baris riwayat transaksi.

Hasil Eksekusi:

- **Algoritma Pintar (KMP):** ~14.00 ms hingga 8.00 ms (Kuning / Merah)
- **Pendekatan Naif (Brute Force JS):** ~9.00 ms hingga 5.00 ms (Hijau / Kuning)

Kesimpulan & Analisis Saintifik:

Berlawanan dengan ekspektasi awal, pendekatan *Brute Force* murni lebih cepat dan lebih stabil. Hal ini terjadi karena:

1. **Karakteristik Data Pendek:** Algoritma KMP (*Knuth-Morris-Pratt*) dirancang untuk mencari frasa di dalam badan teks yang luar biasa panjang (seperti isi buku atau sekuens DNA). Teks transaksi kita sangat pendek (hanya berisi ID Transaksi dan Judul).
2. **Setup Overhead:** KMP memiliki "biaya awal" karena harus menyusun *LPS Table (Longest Prefix Suffix)*. Pada string yang sangat pendek, waktu komputasi pembuatan tabel LPS jauh lebih lama dibandingkan pencocokan string itu sendiri.
3. **Optimasi Mesin Browser (V8 Engine):** Javascript Engine pada peramban moderen telah mengoptimalkan rutinitas pencarian karakter primitif (seperti loop biasa tanpa kondisi bersarang). *Brute Force* memanfaatkan kecepatan primitif ini secara maksimal karena pencarian ID biasanya langsung *mismatch* di karakter pertama, sehingga proses pelompatan (*skip*) terjadi nyaris tanpa beban operasi aritmatika, mengalahkan KMP.

2. Algoritma Pengurutan Merge Sort vs Bubble Sort (Riwayat Teller)

Pengujian dilakukan untuk mengurutkan 5.000 transaksi berdasarkan parameter waktu tempuh (*Timestamp Descending*).

Hasil Eksekusi:

- **Algoritma Pintar (Merge Sort):** Stabil di angka **~0.10 ms - 0.00 ms** (Hijau)
- **Pendekatan Naif (Bubble Sort murni):** Melonjak drastis hingga **> 200.00 ms** (Merah), hingga mengakibatkan sedikit jeda antarmuka (Lag/Freeze).

Kesimpulan & Analisis Saintifik:

Ini adalah pembuktian murni dan absolut dari konsep *Big-O Complexity* di lingkungan nyata.

- *Bubble Sort* berjalan dalam skala kompleksitas $O(n^2)$. Dengan 5.000 elemen, algoritma ini dipaksa melakukan nyaris 25.000.000 operasi perbandingan (hasil dari 5.000×5.000) yang melelahkan.
- *Merge Sort* mengimplementasikan konsep *Divide and Conquer* dengan kompleksitas $O(n \log n)$. 5.000 elemen hanya memerlukan sebagian kecil kalkulasi perbandingan (sekitar 60.000 operasi). Perbedaan skala puluhan juta vs puluhan ribu komputasi ini menjelaskan mengapa layar bisa membeku pada *Bubble Sort*, sementara *Merge Sort* mengeksekusinya dalam pecahan instan tak kasatmata.

3. Algoritma Pecahan Uang Greedy vs Looping Naif (Tarik Tunai Teller)

Pengujian dilakukan saat Teller menarik uang tunai dan sistem otomatis menghitung lembar denominasi uang.

Hasil Eksekusi:

- **Algoritma Pintar (Greedy):** Stabil di **~0.10 ms - 0.00 ms** (Hijau)
- **Pendekatan Naif (Iterasi Pengurangan Berulang):** Stabil di **~0.10 ms - 0.00 ms** (Hijau)

Kesimpulan & Analisis Saintifik:

Meskipun hasil waktunya terlihat sama, hal ini dilatarbelakangi oleh kapabilitas CPU moderen.

- Komputer saat ini dapat menembus eksekusi **1 Miliar instruksi per detik**.
- Untuk pecahan uang misal Rp 1.500.000, metode *Naive* melakukan operasi saldo -= 100000 di dalam *while loop* sebanyak 15 kali. Bagi komputer, mengeksekusi *looping* 15x hanya butuh waktu *micro-second* (di bawah batas pengukuran *millisecond*).

- Namun secara arsitektur matematika, *Greedy* yang menggunakan metode pembagian ($\text{Math.floor}(\text{amount} / \text{pecahan})$) memiliki performa konstan per denominasi alias $\mathcal{O}(m)$ (di mana m adalah jumlah pecahan uang), terlepas dari berapapun besaran uangnya. Sedangkan metode naif akan sangat rapuh jika mata uang yang ditarik bernilai miliaran atau triliunan karena bersifat linier iteratif.

Penutup

Eksperimen yang dilakukan dalam aplikasi ini membuktikan bahwa **tidak semua algoritma tingkat lanjut lebih baik di segala situasi**. Pemilihan algoritma harus selalu menyesuaikan bentuk ekosistem datanya.

- **KMP** luar biasa untuk forensik dokumen raksasa, namun rapuh terhadap *overhead* di data pendek (*micro-string*).
- **Merge Sort** absolut menang telak untuk pengurutan data berskala masif.
- Keduanya sama-sama melambangkan esensi rekayasa perangkat lunak (*Software Engineering*): menukar sedikit kerumitan logika dan memori (*Trade-off*) demi stabilitas sistem tingkat perusahaan.